

# Execution-Aware TinyML Benchmarking for Next-Generation AI-Native Edge Networks

Aaron George  
University of Leeds  
jqt9322@leeds.ac.uk

**Abstract**—TinyML is becoming a key enabler of edge intelligence, allowing resource-constrained devices to perform local inference under latency and quality-of-service constraints. Existing TinyML model-selection methods often rely on architecture-level proxies such as multiply-accumulate operations (MACs), assuming that inference latency is primarily determined by computational complexity. This paper evaluates the reliability of MAC count under realistic execution conditions on an ESP32-S3 microcontroller. Across 90 TinyML models and six execution scenarios, the results show that MAC count remains strongly correlated with mean latency, but is less reliable for tail latency, jitter, and ranking stability under execution stress. In particular, memory pressure produces the highest mean and P99 latency, while combined stress produces the highest latency variability. These findings suggest that MAC count is useful as a first-order efficiency proxy, but should be complemented with execution-aware latency measurements when deployment decisions depend on P99 latency, jitter, or robust model ranking.

## I. INTRODUCTION

The convergence of AI and next-generation wireless networks is increasing the use of resource-constrained edge devices for local inference. TinyML enables such devices to perform inference close to the data source, reducing communication overhead, improving privacy, and supporting real-time decision-making in applications such as wearable monitoring, industrial sensing, and IoT endpoints [1], [2].

A key challenge in this setting is selecting models that satisfy latency, energy, and memory constraints under changing execution conditions. Existing TinyML optimisation and NAS approaches often rely on architecture-level proxies such as multiply-accumulate operations (MACs), parameter count, or latency estimates [3]–[5]. These proxies are useful because they allow large model spaces to be searched efficiently. However, they generally treat inference latency as a largely architecture-dependent quantity and do not fully account for execution-time variability.

This assumption becomes less complete on modern connected microcontrollers. Devices such as the ESP32-S3 execute inference alongside RTOS scheduling, interrupts, wireless activity, memory pressure, and multicore contention. As a result, latency is not only determined by model architecture, but also by the execution environment in which the model runs. MAC count may therefore remain reliable for predicting average latency under stable conditions, while being less reliable for tail latency, jitter, and model-selection decisions under execution stress.

Although TinyML benchmarking has standardised the evaluation of latency, accuracy, and energy under controlled settings, less attention has been given to how latency distributions change under realistic execution conditions. In particular, existing optimisation workflows often treat latency as a single scalar value, rather than a distribution with mean, tail, and variability components. This limits their ability to capture deployment risks in systems where QoS requirements depend not only on average latency, but also on latency stability and worst-case behaviour.

In this work, we evaluate the reliability of MAC count as a TinyML efficiency proxy across different execution conditions. We model inference latency as an execution-dependent random variable  $T(A; E)$ , where  $A$  denotes the model architecture and  $E$  denotes the execution environment. This allows latency to be analysed not only through its mean value, but also through distributional properties such as P99 latency and variability.

We validate this analysis through an empirical study on an ESP32-S3 dual-core microcontroller, evaluating 90 TinyML models across six execution scenarios and collecting over 1.18 million latency measurements. The results show that MAC count is strongly correlated with mean latency, but does not fully capture tail-latency degradation, ranking instability, or variance under stressed execution conditions. These findings motivate execution-aware benchmarking that complements architecture-level proxies with deployment-time latency measurements.

### A. Problem Statement

MAC count is widely used as a proxy for TinyML efficiency, but its reliability depends on both the execution conditions and the latency metric of interest. While it may predict average-case performance under stable conditions, it may not capture tail latency, jitter, or ranking changes caused by wireless activity, interrupts, memory pressure, and multicore contention. This creates a risk that models selected as efficient using architecture-only proxies may not provide the best deployment performance under realistic edge execution.

### B. Research Questions

This paper addresses the following research questions:

- Under which execution conditions does MAC count reliably predict TinyML inference latency?
- How do execution stressors affect mean latency, P99 latency, jitter, and model-ranking stability?

- Which architectural properties are most strongly associated with latency variability?

### C. Contributions

This paper makes two main contributions:

- **Conditional evaluation of MAC-count reliability:** We analyse when MAC count is a reliable predictor of TinyML inference performance and when it becomes insufficient. The study distinguishes between average latency, tail latency, jitter, and ranking stability across multiple execution environments.
- **Execution-aware empirical study on a connected microcontroller:** We evaluate 90 TinyML models across six execution scenarios on an ESP32-S3, collecting over 1.18 million latency measurements. The results show how wireless activity, interrupts, memory pressure, and multicore contention affect latency distributions and proxy-based model-selection decisions.

## II. LITERATURE REVIEW AND BACKGROUND

TinyML research has focused extensively on optimising neural networks for deployment on resource-constrained microcontrollers [1], [2]. Early frameworks such as MCUNet demonstrated that deep neural networks can be co-designed with hardware constraints through compression, quantisation, and memory-aware scheduling to enable efficient on-device execution [3]. Similarly, MicroNets explored architectural trade-offs between accuracy and efficiency, primarily guided by proxy-based optimisation strategies.

Neural Architecture Search (NAS) techniques in TinyML rely heavily on computational proxies such as multiply-accumulate (MAC) count, parameter count, and latency lookup tables [4]–[6]. These proxies enable efficient exploration of large architectural search spaces, but implicitly assume a stable and predictable relationship between computational cost and runtime performance. Consequently, model selection is typically performed without direct consideration of execution dynamics or system-level variability.

Benchmarking efforts such as MLPerf Tiny have standardised evaluation methodologies for TinyML systems, enabling reproducible comparisons across models and platforms [7]. These benchmarks focus on metrics such as latency, accuracy, and energy consumption under controlled conditions. While valuable, they largely assume deterministic execution environments and therefore do not capture the variability encountered in real-world AI-native edge deployments, where interference and contention are structural, not incidental [8].

In contrast, real-time systems research has long recognised the importance of execution variability. Concepts such as worst-case execution time (WCET), probabilistic timing analysis, and scheduling interference explicitly model dynamic system behaviour and its impact on timing guarantees [9]–[11]. However, these principles have not been systematically integrated into TinyML model evaluation or NAS workflows, which continue to rely on static, architecture-centric metrics.

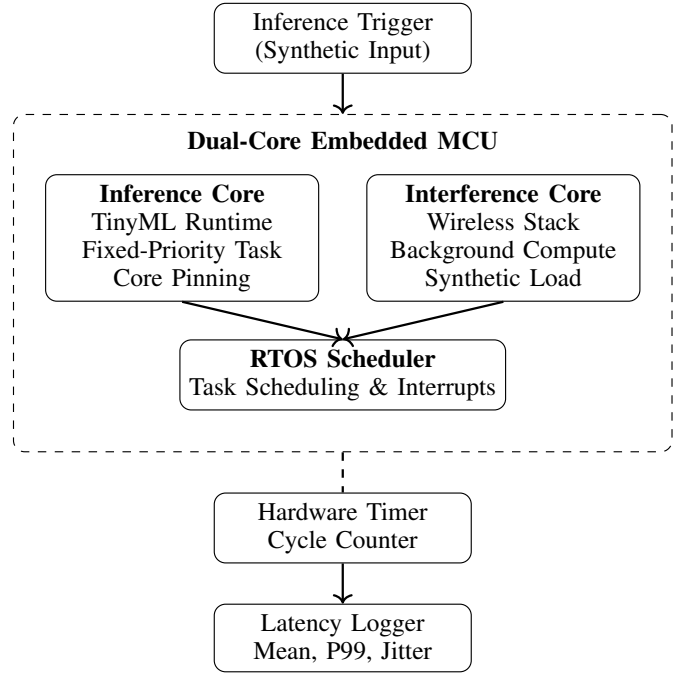


Fig. 1. System model for execution-aware TinyML evaluation. Inference executes on a dedicated core under a fixed-priority task, while controlled interference is introduced on the second core via a wireless stack and background compute tasks, scheduled by an RTOS. Latency is captured using hardware timers and logged for distributional analysis.

More recent work has begun to consider system-level factors influencing inference performance, including memory access patterns, cache behaviour, and hardware-specific optimisations [8], [12], [13]. These studies highlight that performance is influenced by factors beyond raw computational complexity. However, they are typically limited in scope, and do not provide a unified framework for understanding how execution conditions interact with model structure to determine performance.

## III. SYSTEM ARCHITECTURE

This section describes the system model used to evaluate execution-aware efficiency metrics under realistic embedded inference conditions. The design is centred around a dual-core embedded microcontroller where TinyML inference executes on a dedicated core while sources of controlled interference are introduced on the second core to simulate stochastic execution semantics representative of AI-native edge deployments.

The inference pipeline consists of four stages: input acquisition, preprocessing, model inference, and output generation. Synthetic inputs are used throughout to eliminate variability caused by data-dependent execution, ensuring that observed latency differences are attributable to architectural and environmental factors alone.

### A. Execution Scenarios

Six scenarios are defined to move from isolated execution to combined stress. Table I summarises the environment set  $\mathcal{E}$ .

TABLE I  
EXECUTION SCENARIOS USED FOR BENCHMARKING

| Scenario | Description                                                      |
|----------|------------------------------------------------------------------|
| E0       | Idle baseline with no deliberate background interference.        |
| E1       | Interrupt-heavy execution with frequent asynchronous events.     |
| E2       | Active wireless communication causing bursty system activity.    |
| E3       | Multi-core or background compute contention.                     |
| E4       | Memory-pressure scenario stressing activation and buffer access. |
| E5       | Combined stress with all interference sources active.            |

Each scenario corresponds to a different regime of the execution process. In deployment, an edge node may transition among these regimes depending on packet arrivals, sensing frequency, service load, and background tasks. Therefore, a robust model-selection method should not depend only on ideal baseline latency.

#### IV. EFFICIENCY AS A FUNCTION OF ARCHITECTURE AND EXECUTION SEMANTICS

To better understand the relationship between model design and system conditions, we introduce a formal framework for execution-aware efficiency. Let  $\mathcal{A}$  denote the space of architectures, where each architecture  $A \in \mathcal{A}$  is defined by its structure (layers, connectivity, and parameters). Let  $\mathcal{E}$  denote the space of execution environments, where  $E \in \mathcal{E}$  represents a particular system state, including scheduling conditions, interrupts, and resource contention.

For a given architecture  $A$  deployed under execution environment  $E$ , the inference latency is modelled as a random variable:

$$T(A; E), \quad (1)$$

which captures the stochastic nature of execution. Rather than being a fixed scalar, latency is characterised by a distribution, from which we can derive statistics such as the mean  $\mathbb{E}[T(A; E)]$ , variance, and high-percentile latency  $T_p(A; E)$ . This formulation emphasises that latency depends jointly on architecture and execution conditions not on architecture alone and is therefore the natural basis for model selection in AI-native edge systems where execution conditions vary continuously.

##### A. Conditional Rank Stability

NAS pipelines often rely on proxy metrics  $p(A)$ , such as MAC count, to rank architectures without direct deployment measurement. These proxies are useful when the ranking induced by  $p(A)$  is consistent with measured latency under the target execution condition.

**Definition (Rank Preservation).** A proxy metric  $p(A)$  is rank-preserving under execution environment  $E$  if, for any architectures  $A, B \in \mathcal{A}$ :

$$p(A) < p(B) \implies \mathbb{E}[T(A; E)] < \mathbb{E}[T(B; E)]. \quad (2)$$

In practice, rank preservation may hold approximately for mean latency under stable execution, but can weaken when tail latency, jitter, or stressed execution conditions are considered.

This occurs because architectures with similar computational cost may differ in memory footprint, activation lifetime, tensor reuse, and sensitivity to interrupts or shared-resource contention. Therefore, MAC count should be treated as a conditional proxy: useful for coarse average-latency estimation, but not sufficient as a complete deployment metric.

##### B. Stochastic Model of Execution Semantics

To explicitly capture execution variability, we model execution semantics as a stochastic process  $\{E_t\}$  evolving over time. Each state  $E_t$  represents system-level conditions such as idle execution, interrupt-heavy workload, network activity (Wi-Fi/BLE), or resource contention. Conditioned on the execution state, the latency of architecture  $A$  follows a state-dependent distribution:

$$T_t(A) \mid E_t = e \sim D_{A,e}, \quad (3)$$

where  $D_{A,e}$  is the latency distribution under execution state  $e$ .

The overall latency distribution  $T(A; E)$  arises from the mixture induced by the stochastic evolution of  $\{E_t\}$ . This formulation captures temporal variability, burstiness, and correlation in execution behaviour, all characteristic of AI-native edge nodes whose execution environment is shaped by unpredictable wireless traffic and service arrivals. Crucially, different architectures respond differently to the same execution process, so performance ordering depends on both the architecture and the statistical properties of the execution environment.

##### C. Execution-Aware Efficiency Metric

To address the limitations of proxy metrics, we define an execution-aware efficiency metric:

$$J(A; E) = \alpha \mathbb{E}[T(A; E)] + \beta T_{99}(A; E) + \gamma \sigma(T(A; E)), \quad (4)$$

where  $\mathbb{E}[T(A; E)]$  is the mean latency,  $T_{99}(A; E)$  is the 99th-percentile latency,  $\sigma(T(A; E))$  is the standard deviation (jitter), and  $\alpha, \beta, \gamma \geq 0$  are application-specific weighting coefficients.

This metric captures three key aspects of performance: average performance through mean latency, tail behaviour through P99 latency, and stability through variability. Unlike proxy metrics,  $J(A; E)$  is explicitly dependent on execution conditions, making it suitable for deployment-aware optimisation in AI-native edge contexts where QoS targets span all three dimensions.

##### D. Design Principles for Execution-Aware Metrics

An effective efficiency metric should satisfy the following properties:

- **Execution Dependence:** Efficiency must depend on both architecture and execution environment.
- **Tail Sensitivity:** Metrics must account for latency spikes and variability, not just averages.
- **Dominance Consistency:** If one architecture outperforms another across all performance dimensions, it should have a strictly better efficiency score.

- **Deployment Relevance:** Metrics should correlate with real-world system performance.

The proposed metric satisfies these properties and provides a principled basis for execution-aware TinyML optimisation in AI-native edge networks.

## V. METHODOLOGY

### A. Hardware Platform

The experimental platform is an ESP32-S3 dual-core microcontroller operating at 240 MHz. The device includes two Xtensa LX7 cores, integrated Wi-Fi and BLE connectivity, and support for external PSRAM, providing a practical platform for evaluating TinyML inference on resource-constrained connected edge devices. In all experiments, the inference task was pinned to Core 0, while Core 1 was used to generate controlled interference. This setup allows the inference workload to be measured under repeatable baseline and stressed execution conditions, while capturing system-level effects caused by wireless activity, interrupts, background computation, and memory contention.

### B. Model Dataset

To ensure comprehensive coverage of the TinyML design space, we construct a dataset of 90 neural network models spanning multiple architecture families, including depthwise-separable convolution variants, residual designs, and lightweight convolution blocks. These models are selected to capture a wide range of architectural characteristics, including variations in depth, width, parameter count, and activation footprint.

The dataset includes both lightweight models designed for ultra-low-power applications and more complex architectures that approach the computational and memory limits of embedded devices. This diversity enables systematic analysis of how architectural properties influence execution behaviour under varying system conditions representative of AI-native edge deployments.

### C. Measurement Protocol

We collect a total of 1.18 million latency measurements across all models and execution scenarios. Each model-scenario pair is evaluated using a minimum of 1000 inference runs to ensure statistically reliable estimates of latency distributions.

Latency is measured using hardware timers with microsecond resolution, minimising instrumentation overhead and ensuring high measurement accuracy. The system operates at a fixed clock frequency to eliminate variability introduced by dynamic frequency scaling.

Inference tasks are pinned to a dedicated core to isolate execution, while controlled background interference is introduced to simulate realistic system conditions. All experiments are conducted under consistent configurations to ensure reproducibility across runs.

TABLE II  
SUMMARY OF KEY EMPIRICAL FINDINGS.

| Finding                          | Observed Value                |
|----------------------------------|-------------------------------|
| MAC vs. mean latency correlation | Spearman $\rho \approx 0.975$ |
| Maximum mean-latency rank shift  | 8 positions                   |
| Maximum P99-latency rank shift   | 9 positions                   |
| Maximum jitter-rank shift        | 76 positions                  |
| Idle jitter under E0             | 0.003 ms                      |
| Maximum jitter under E5          | 1.281 ms                      |
| Highest mean-latency scenario    | E4 memory pressure            |
| Matched-model latency divergence | Up to $6.4\times$             |

TABLE III  
LATENCY BEHAVIOUR ACROSS EXECUTION SCENARIOS. VALUES ARE AVERAGED ACROSS THE 89 MODELS COMMON TO ALL SCENARIOS. JITTER IS THE MEAN WITHIN-MODEL STANDARD DEVIATION.

| Scenario | Mean (ms) | P99 (ms) | Jitter (ms) | P99 Overhead (ms) |
|----------|-----------|----------|-------------|-------------------|
| E0       | 757.77    | 757.77   | 0.003       | 0.005             |
| E1       | 763.92    | 763.93   | 0.006       | 0.013             |
| E2       | 788.92    | 793.18   | 1.231       | 4.25              |
| E3       | 852.62    | 852.64   | 0.010       | 0.022             |
| E4       | 1108.77   | 1109.41  | 0.277       | 0.636             |
| E5       | 941.59    | 944.25   | 1.281       | 2.66              |

### D. Feature Extraction

To analyse the relationship between architecture and execution variability, we extend the study to a larger dataset of 280 models. For each model, we extract a set of architectural features capturing both computational and memory-related characteristics, including:

- Total parameter count
- Peak activation memory footprint
- Number of convolutional layers (depth)
- Maximum number of filters (width)
- Parameter density per layer
- Activation density

These features are used to perform correlation analysis and identify the primary drivers of latency variance, distinguishing between compute-related and memory-related factors.

## VI. RESULTS

### A. Overview

The results show that MAC count remains a strong predictor of mean latency, but is less reliable for tail latency, jitter, and ranking stability under stressed execution. Table II summarises the main findings.

### B. Latency Across Execution Scenarios

Table III reports latency behaviour across the six execution scenarios, averaged over the 89 models common to all scenarios. E4 produces the highest mean and P99 latency, while E5 produces the highest jitter. This shows that different stressors affect different aspects of latency: memory pressure mainly increases runtime cost, while combined stress mainly increases variability.

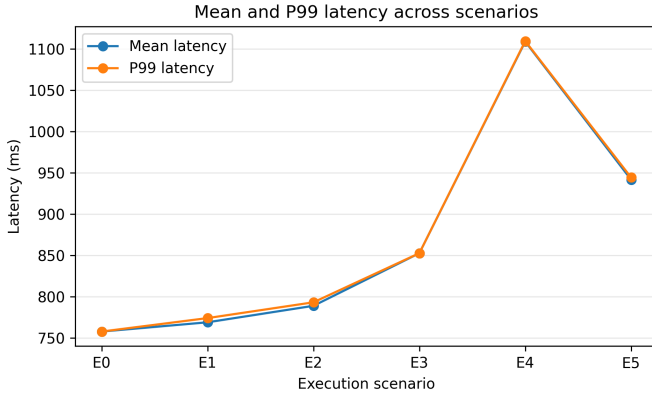


Fig. 2. Mean and P99 latency across execution scenarios. E4 produces the highest mean and P99 latency, indicating that memory pressure has the strongest effect on average runtime cost.

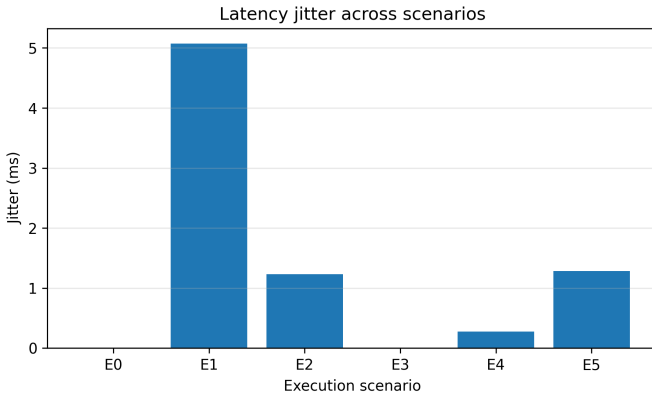


Fig. 3. Latency jitter across execution scenarios. Combined stress (E5) produces the highest jitter, showing that execution stress affects latency predictability as well as latency magnitude.

Compared with E0, E5 increases jitter from 0.003 ms to 1.281 ms, while E4 increases mean latency from 757.77 ms to 1108.77 ms. Therefore, execution stress affects not only latency magnitude but also latency predictability.

### C. Proxy Correlation and Ranking Stability

MAC count has a strong correlation with mean latency, with Spearman  $\rho \approx 0.975$ . This confirms that MAC count is useful as a first-order proxy for average-case performance. However, this does not guarantee stable rankings under stress. Models shift by up to 8 positions in mean-latency ranking and up to 9 positions in P99-latency ranking, while jitter rankings shift by up to 76 positions.

These shifts matter because TinyML search pipelines often retain only a small top-ranked candidate set. A model that appears efficient under a proxy metric may therefore be less suitable once tail latency or jitter is considered.

### D. Tail Latency Across Model Families

Table IV shows mean P99 latency for representative model families under E0, E2, and E5. All families show higher P99 latency under stressed execution.

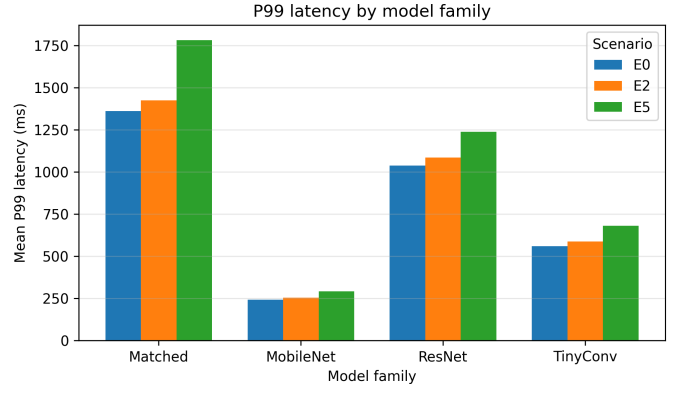


Fig. 4. Mean P99 latency by model family under selected execution scenarios. All model families show increased P99 latency under stressed execution, although the magnitude of degradation differs by architecture family.

TABLE IV  
MEAN P99 LATENCY BY MODEL FAMILY UNDER SELECTED EXECUTION SCENARIOS.

| Model Family | E0 P99 (ms) | E2 P99 (ms) | E5 P99 (ms) |
|--------------|-------------|-------------|-------------|
| Matched      | 1360.38     | 1423.23     | 1781.05     |
| MobileNet    | 241.37      | 254.67      | 291.86      |
| ResNet       | 1038.15     | 1084.29     | 1238.50     |
| TinyConv     | 632.57      | 662.37      | 779.13      |

MobileNet has the lowest P99 latency across all scenarios, while the Matched family has the highest. However, the consistent increase from E0 to E2 and E5 shows that execution conditions affect tail latency across all model families.

### E. Architectural Factors Associated with Variability

Table V reports correlations between architectural features and latency variance. Activation memory footprint and parameter density have the strongest correlations, while MAC count and network depth are weakly correlated.

This suggests that latency variability is more closely associated with memory-related properties than with arithmetic complexity alone. MAC count is therefore useful for mean-latency estimation, but should be complemented with execution-aware measurements when tail latency and jitter matter.

### F. Implications for Model Selection

Overall, MAC count is conditionally reliable. It is effective for coarse filtering and average-latency estimation, but insufficient for final deployment decisions involving P99 latency, jitter, or stressed execution. A practical approach is therefore to use MAC count during early model search, followed by execution-aware benchmarking of shortlisted models.

## VII. FUTURE DIRECTIONS

- **Surrogate Models for NAS:** Develop lightweight predictors of  $J(A; E)$  using architectural features to enable execution-aware search without full deployment.
- **Scheduling Co-Design:** Explore co-optimisation of model architectures and RTOS scheduling policies to

TABLE V  
CORRELATION BETWEEN ARCHITECTURAL FEATURES AND LATENCY VARIANCE.

| Feature                     | Correlation with Variance ( $\rho$ ) |
|-----------------------------|--------------------------------------|
| Activation memory footprint | <b>0.82</b>                          |
| Parameter density           | <b>0.76</b>                          |
| Total parameters            | 0.41                                 |
| MAC count                   | 0.12                                 |
| Network depth               | 0.09                                 |

jointly minimise latency variability and mean inference cost.

- **Robust Architecture Design:** Incorporate execution-robustness objectives directly into NAS search spaces, targeting AI-native edge QoS requirements.
- **Cross-Platform Validation:** Extend the evaluation framework to additional MCU families to assess the generality of execution-dependent effects.
- **Energy-Aware Extension:** Integrate power and energy measurements to study variability across both latency and energy dimensions under dynamic execution conditions.

### VIII. CONCLUSION

This work evaluates the reliability of MAC count as a TinyML efficiency proxy under different edge execution conditions. The results show that MAC count remains useful for predicting average inference latency, but is less reliable for tail latency, jitter, and ranking stability under stressed execution.

Experiments on an ESP32-S3 across 90 TinyML models and six execution scenarios show that different stressors affect latency in different ways. Memory pressure produces the highest mean and P99 latency, while combined stress produces the highest jitter. Models also shift by up to 8 positions in mean-latency ranking and up to 9 positions in P99-latency ranking, indicating that proxy-based rankings can become less stable under realistic execution conditions.

Overall, the findings support a conditional view of MAC count: it is effective as a first-order proxy for coarse model filtering, but should be complemented with execution-aware benchmarking when deployment decisions depend on P99 latency, jitter, or robust QoS behaviour.

### REFERENCES

- [1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2019.
- [2] M. Rusci, D. Rossi, and L. Benini, "A survey on TinyML: Applications, challenges and future directions," *IEEE Embedded Systems Letters*, 2021.
- [3] J. Lin, W.-M. Chen, Y. Lin, C. Gan, and S. Han, "MCUNet: Tiny deep learning on IoT devices," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [4] Q. Zhou *et al.*, " $\mu$ NAS: Constrained neural architecture search for microcontrollers," in *IEEE International Conference on Computer Design (ICCD)*, 2021.
- [5] H. Cai, L. Zhu, and S. Han, "ProxylessNAS: Direct neural architecture search on target task and hardware," in *International Conference on Learning Representations (ICLR)*, 2019.
- [6] T. Elsken, J. H. Metzen, and F. Hutter, "Neural architecture search: A survey," *Journal of Machine Learning Research*, 2019.
- [7] C. Banbury *et al.*, "MLPerf tiny benchmark," in *Proceedings of Machine Learning and Systems (MLSys)*, 2021.
- [8] Y. Zhao *et al.*, "Understanding latency variability in deep learning inference," *IEEE Micro*, 2022.
- [9] R. Pellizzoni, M. Caccamo, and L. Sha, "Memory interference analysis in multicore systems," in *IEEE Real-Time Systems Symposium (RTSS)*, 2010.
- [10] S. Altmeyer *et al.*, "Cache-related preemption delay analysis," in *Euro-micro Conference on Real-Time Systems (ECRTS)*, 2011.
- [11] G. Bernat, A. Colin, and S. M. Petters, "Probabilistic timing analysis for real-time systems," in *IEEE Real-Time Systems Symposium (RTSS)*, 2002.
- [12] S. Williams *et al.*, "Roofline: An insightful visual performance model," *Communications of the ACM*, 2009.
- [13] M. Chen *et al.*, "Eyeriss: An energy-efficient reconfigurable accelerator for CNNs," *IEEE Journal of Solid-State Circuits*, 2016.